

DEEP LEARNING AND APPLICATIONS

MR20-1CS0158

UNIT III

Prepared by

Dr.M.Narayanan

Professor

Department of CSE

Malla Reddy University, Hyderabad

DEEP LEARNING AND APPLICATIONS

MR20-1CS0158

UNIT III

Artificial Neural Networks: Introduction, Perceptron Training Rule, Gradient Descent Rule.

Gradient Descent and Back propagation: Gradient Descent, Stochastic Gradient Descent, Back propagation, Some problems in ANN Optimization and Regularization: Over fitting and Capacity, Cross Validation, Feature Selection, Regularization, Hyper parameters.

Text Book

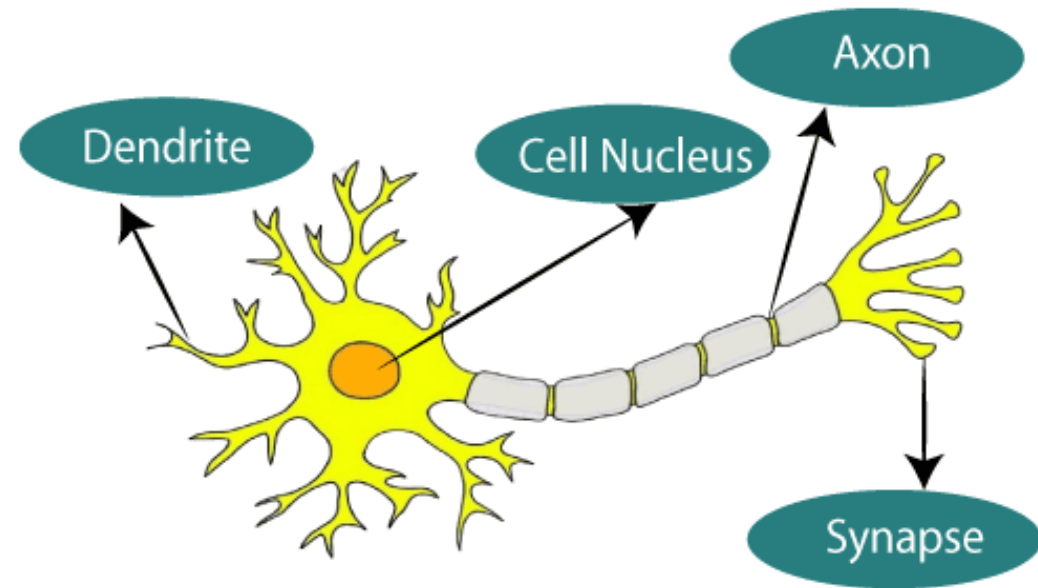
1. Goodfellow, I., Bengio, Y., and Courville, A., Deep Learning, MIT Press, 2016

Artificial Neural Networks: Introduction

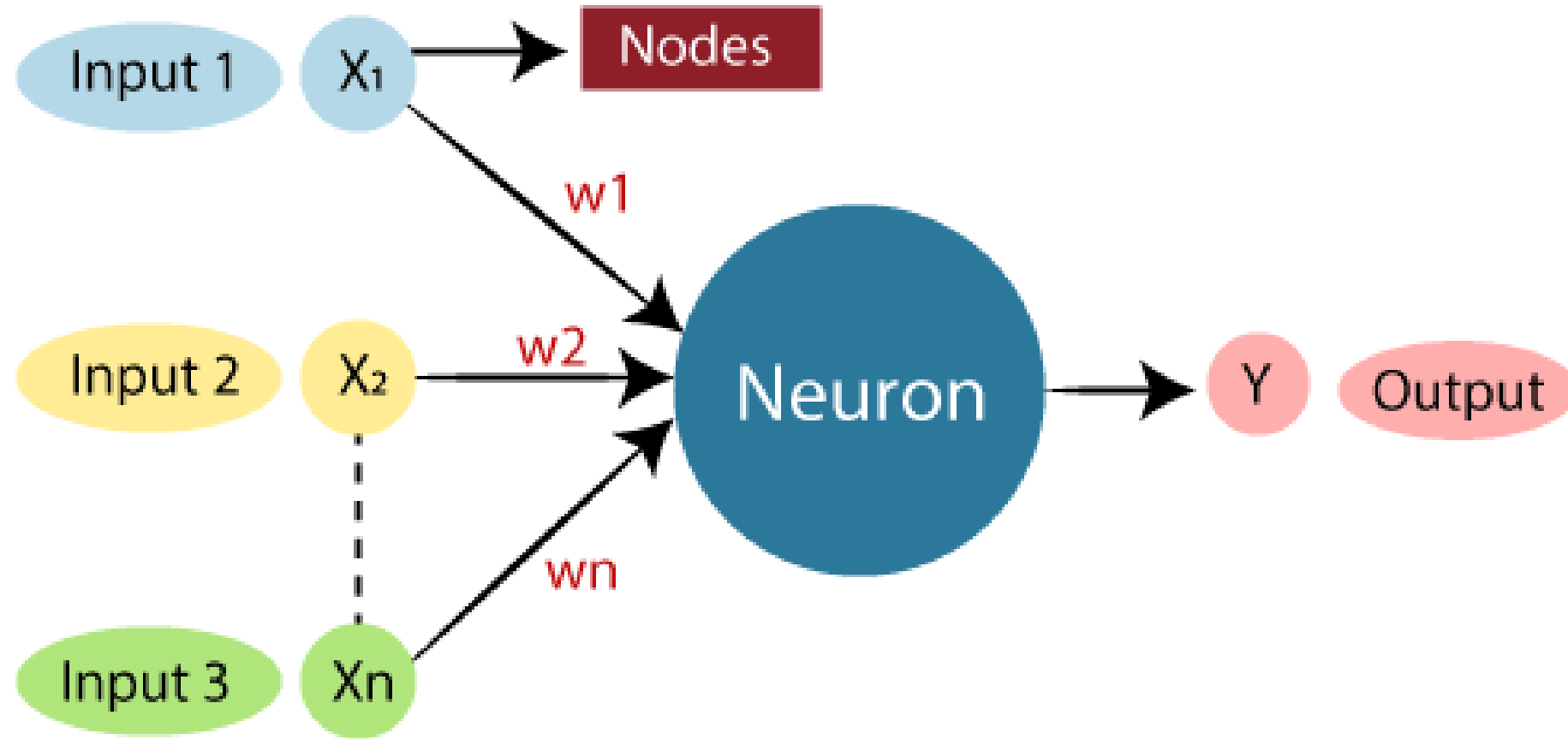
What is Artificial Neural Network?

- ❖ The term "Artificial Neural Network" is derived from Biological neural networks that develop the structure of a human brain.
- ❖ Similar to the human brain that has neurons interconnected to one another, artificial neural networks also have neurons that are interconnected to one another in various layers of the networks. These neurons are known as nodes.

- ❖ The given figure illustrates the typical diagram of Biological Neural Network.



The typical Artificial Neural Network looks something like the given figure.



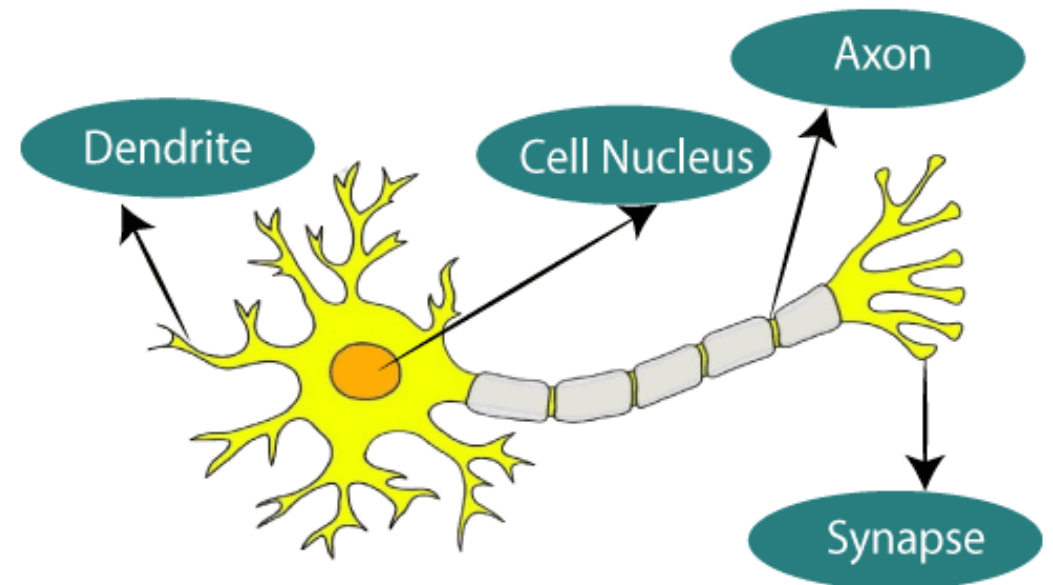
- ❖ **Dendrites** from Biological Neural Network represent **inputs** in Artificial Neural Networks, **cell nucleus** represents **Nodes**, **synapse** represents **Weights**, and **Axon** represents **Output**.
- ❖ Relationship between Biological neural network and artificial neural network:

Biological Neural Network

Dendrites
Cell nucleus
Synapse
Axon

Artificial Neural Network

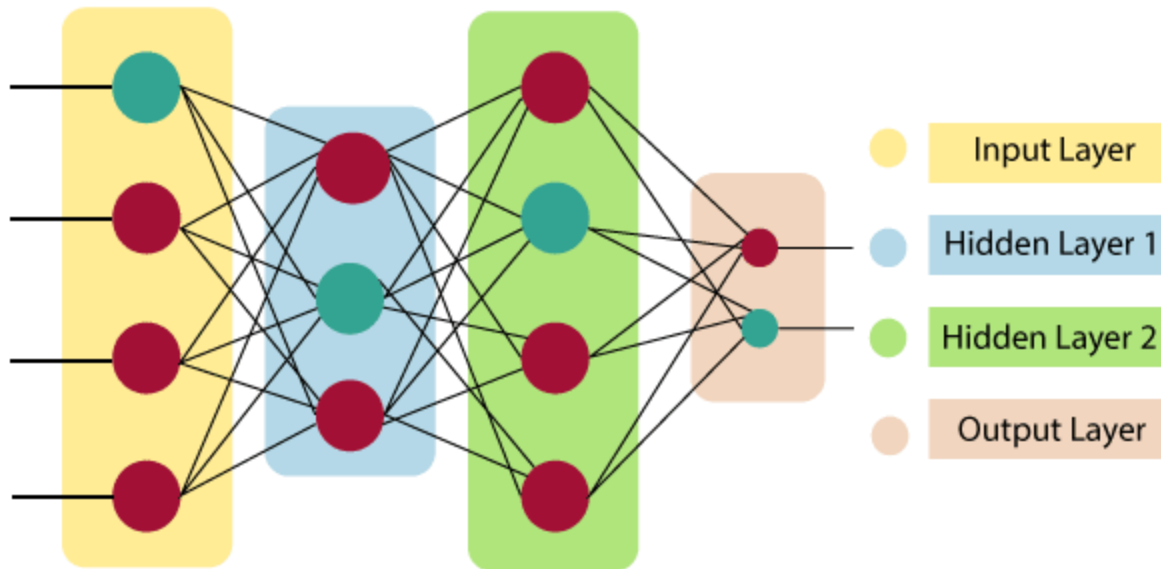
Inputs
Nodes
Weights
Output



- ❖ An Artificial Neural Network in the field of Artificial intelligence where it attempts to mimic the network of neurons makes up a human brain so that computers will have an option to understand things and make decisions in a human-like manner.
- ❖ **The artificial neural network is designed by programming computers to behave simply like interconnected brain cells.**
- ❖ There are around 1000 billion neurons in the human brain.
- ❖ Each neuron has an association point somewhere in the range of 1,000 and 100,000.
- ❖ In the human brain, data is stored in such a manner as to be distributed, and we can extract more than one piece of this data when necessary from our memory parallelly.
- ❖ We can say that the human brain is made up of extraordinarily **amazing parallel processors**.

The architecture of an artificial neural network:

- ❖ To understand the concept of the architecture of an artificial neural network, we have to understand what a neural network consists of. In order to define a neural network that consists of a large number of artificial neurons, which are termed units arranged in a sequence of layers. Lets us look at various types of layers available in an artificial neural network.
- ❖ Artificial Neural Network primarily consists of three layers:



Input Layer:

- ❖ As the name suggests, it accepts inputs in several different formats provided by the programmer.

Hidden Layer:

- ❖ The hidden layer presents in-between input and output layers. It performs all the calculations to find hidden features and patterns.

Output Layer:

- ❖ The input goes through a series of transformations using the hidden layer, which finally results in output that is conveyed using this layer.
- ❖ The artificial neural network takes input and computes the weighted sum of the inputs and includes a bias. This computation is represented in the form of a transfer function.

$$\sum_{i=1}^n W_i * X_i + b$$

- ❖ It determines weighted total is passed as an input to an activation function to produce the output. Activation functions choose whether a node should fire or not.
- ❖ Only those who are fired make it to the output layer.
- ❖ There are distinctive activation functions available that can be applied upon the sort of task we are performing.

Advantages of Artificial Neural Network (ANN)

Parallel processing capability:

- ❖ Artificial neural networks have a numerical value that can perform more than one task simultaneously.

Storing data on the entire network:

- ❖ Data that is used in traditional programming is stored on the whole network, not on a database. The disappearance of a couple of pieces of data in one place doesn't prevent the network from working.

Capability to work with incomplete knowledge:

- ❖ After ANN training, the information may produce output even with inadequate data. The loss of performance here relies upon the significance of missing data.

Having a memory distribution:

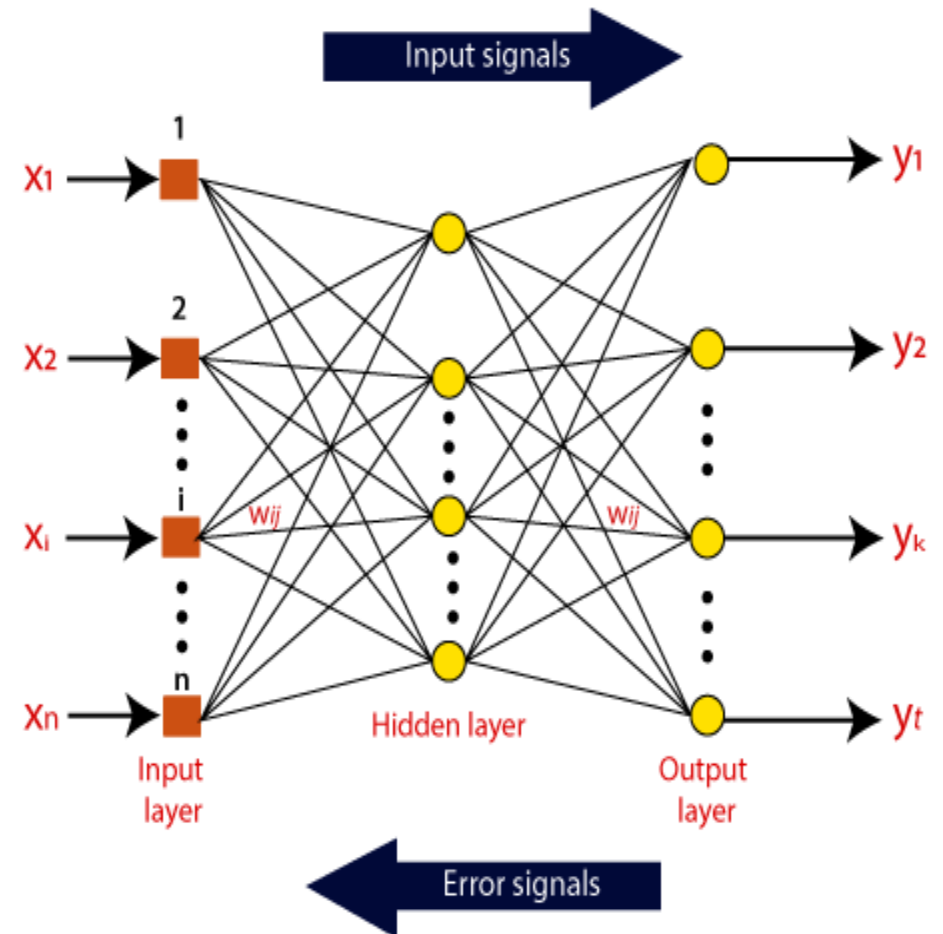
- ❖ For ANN is to be able to adapt, it is important to determine the examples and to encourage the network according to the desired output by demonstrating these examples to the network.
- ❖ The succession of the network is directly proportional to the chosen instances, and if the event can't appear to the network in all its aspects, it can produce false output.

Having fault tolerance:

- ❖ Extortion of one or more cells of ANN does not prohibit it from generating output, and this feature makes the network fault-tolerance.

How do artificial neural networks work?

- ❖ Artificial Neural Network can be best represented as a weighted directed graph, where the artificial neurons form the nodes. The association between the neurons outputs and neuron inputs can be viewed as the directed edges with weights.
- ❖ The Artificial Neural Network receives the input signal from the external source in the form of a pattern and image in the form of a vector.
- ❖ These inputs are then mathematically assigned by the notations $x(n)$ for every n number of inputs.



- ❖ Afterward, each of the input is multiplied by its corresponding weights (these weights are the details utilized by the artificial neural networks to solve a specific problem).
- ❖ In general terms, these weights normally represent the strength of the interconnection between neurons inside the artificial neural network.
- ❖ All the weighted inputs are summarized inside the computing unit.
- ❖ If the weighted sum is equal to zero, then bias is added to make the output non-zero or something else to scale up to the system's response.
- ❖ Bias has the same input, and weight equals to 1.
- ❖ Here the total of weighted inputs can be in the range of 0 to positive infinity.
- ❖ Here, to keep the response in the limits of the desired value, a certain maximum value is benchmarked, and the total of weighted inputs is passed through the activation function.

- ❖ The activation function refers to the set of transfer functions used to achieve the desired output. There is a different kind of the activation function, but primarily either linear or non-linear sets of functions.
- ❖ Some of the commonly used sets of activation functions are the Binary, linear, and Tan hyperbolic, sigmoidal activation functions. Let us take a look at each of them in details.
- ❖ In binary activation function, the output is either a one or a 0. Here, to accomplish this, there is a threshold value set up.
- ❖ If the net weighted input of neurons is more than 1, then the final output of the activation function is returned as one or else the output is returned as 0.

Types of Artificial Neural Network:

- ❖ There are various types of Artificial Neural Networks (ANN) depending upon the human brain neuron and network functions, an artificial neural network similarly performs tasks. The majority of the artificial neural networks will have some similarities with a more complex biological partner and are very effective at their expected tasks. For example, segmentation or classification.

Feedback ANN:

- ❖ In this type of ANN, the output returns into the network to accomplish the best-evolved results internally. As per the University of Massachusetts, Lowell Centre for Atmospheric Research. The feedback networks feed information back into itself and are well suited to solve optimization issues. The Internal system error corrections utilize feedback ANNs.

Feed-Forward ANN:

- ❖ A feed-forward network is a basic neural network comprising of an input layer, an output layer, and at least one layer of a neuron.
- ❖ Through assessment of its output by reviewing its input, the intensity of the network can be noticed based on group behavior of the associated neurons, and the output is decided.
- ❖ The primary advantage of this network is that it figures out how to evaluate and recognize input patterns.

Disadvantages of Artificial Neural Network:

Assurance of proper network structure:

- ❖ There is no particular guideline for determining the structure of artificial neural networks. The appropriate network structure is accomplished through experience, trial, and error.

Unrecognized behavior of the network:

- ❖ It is the most significant issue of ANN. When ANN produces a testing solution, it does not provide insight concerning why and how. It decreases trust in the network.

Hardware dependence:

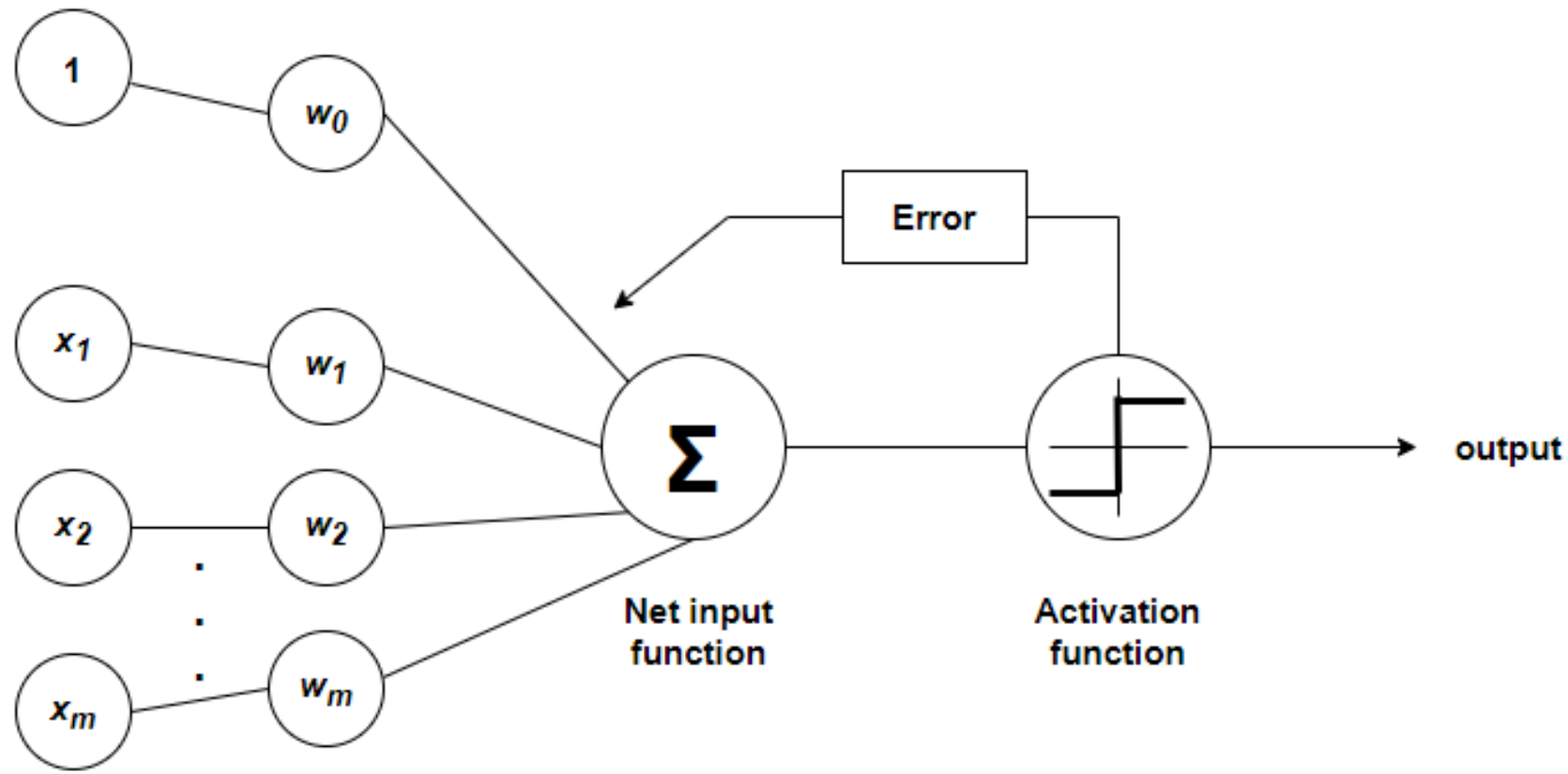
- ❖ Artificial neural networks need processors with parallel processing power, as per their structure. Therefore, the realization of the equipment is dependent.

Difficulty of showing the issue to the network:

- ❖ ANNs can work with numerical data. Problems must be converted into numerical values before being introduced to ANN. The presentation mechanism to be resolved here will directly impact the performance of the network. It relies on the user's abilities.

Perceptron learning rule

- ❖ A perceptron takes inputs, modifies the inputs using certain weights, and then uses a function to output the final result. This is shown in the diagram below



Perceptron learning rule

The output generated is based on the input values, $x_1, x_2, \dots, x_m$. The output can only have two values (binary classification), usually **1** or **0**.

The summation function (represented by Σ in the above diagram) multiplies the inputs with the weights and then adds them up. This can be represented using the following equation:

$$w_0 + w_1x_1 + w_2x_2 + \dots + w_mx_m$$

$$o(x_1, x_2, x_3, \dots, x_m) = \begin{cases} 1 & \text{if } w_0 + w_1x_1 + w_2x_2 + \dots + w_mx_m > 0 \\ -1 & \text{otherwise} \end{cases}$$

A simple activation function

Activation function

- ❖ The activation function converts the numerical output of the weighting function to 1 or 0.
- ❖ The following is an example of a simple activation function:

Error and adjustments

- ❖ Before we can use a perceptron to predict output, it is trained using labeled data. For each input in the training set, we compute the output. If the observed output does not match the expected output, we calculate the error.
- ❖ Initially, we usually set the weights to randomly selected numbers.
- ❖ We can then use the error to tweak the weights in favor of the expected output. We repeat this process until the perceptron gives a high degree of accuracy in its output.
- ❖ How much we adjust our weights is controlled by the learning rate of the perceptron.

Perceptron Training Algorithm

- ❖ To find a set of weights and a bias that allows a perceptron to correctly classify a set of linearly separable input patterns into two classes.

Inputs:

- ❖ A set of training examples: $\{(x_1, t_1), (x_2, t_2), \dots, (x_P, t_P)\}$
- ❖ x_p : An input vector for the p -th example (e.g., $[x_{p1}, x_{p2}, \dots, x_{pN}]$, where N is the number of input features).
- ❖ t_p : The target output (desired class label) for the p -th example, typically -1 or 1 for binary classification.
- ❖ Learning rate η (eta): A small positive constant, usually between 0.0 and 1.0 (e.g., 0.1).

Outputs:

- ❖ A set of optimal weights $w = [w_1, w_2, \dots, w_N]$
- ❖ An optimal bias b

Algorithm Steps:

Step 1: Initialization

- ❖ **Initialize Weights (w):** Set all connection weights w_i (for $i = 1$ to N) to small random values, often 0. Alternatively, you can initialize them to all zeros. For simplicity in theoretical examples, all zeros is common.
- ❖ **Initialize Bias (b):** Set the bias b to a small random value, often 0.
- ❖ **Set Learning Rate (η):** Choose a suitable learning rate, e.g., $\eta = 0.1$.

Step 2: Define Activation Function (Activation Rule)

- ❖ The perceptron typically uses a step (or sign/bipolar) activation function. For a calculated net input S :
 - ❖ If $S \geq 0$, then the output $y = 1$.
 - ❖ If $S < 0$, then the output $y = -1$.

Step 3: Training Loop (Epochs)

- ❖ Loop: Repeat the following steps for a fixed number of epochs (iterations over the entire training dataset) or until the network converges (no misclassifications occur for an entire epoch).

Step 4: Iterate Through Training Examples

- ❖ For each training example p from 1 to P :
 - ❖ Let the current input vector be $x_p = [x_{p1}, x_{p2}, \dots, x_{pN}]$.
 - ❖ Let the current target output be t_p .

Step 5: Calculate Net Input (S)

- ❖ Compute the net input (or weighted sum) for the perceptron's output neuron:
- ❖ $S = b + (x_{p1} * w_1) + (x_{p2} * w_2) + \dots + (x_{pN} * w_N)$
- ❖ This can be written in vector form as:
- ❖ $S = b + x_p \cdot w$ (where $\cdot$ denotes the dot product)

Step 6: Calculate Actual Output (y)

- ❖ Apply the activation function (from Step 2) to the net input S to get the perceptron's actual output y :
 - ❖ If $S \geq 0$, then $y = 1$.
 - ❖ If $S < 0$, then $y = -1$.

Step 7: Update Weights and Bias (Perceptron Learning Rule)

- ❖ **Check for Error:** Compare the actual output y with the target output t_p .
- ❖ If $y \neq t_p$ (Error occurred):
 - ❖ **Calculate Error Term (δ):** $\delta = t_p - y$
 - ❖ Note: If $t_p = 1$ and $y = -1$, $\delta = 1 - (-1) = 2$.
 - ❖ If $t_p = -1$ and $y = 1$, $\delta = -1 - 1 = -2$.
 - ❖ **Update Weights:** For each weight w_i :
 - ❖ $w_i(\text{new}) = w_i(\text{old}) + \eta * \delta * x_{pi}$
 - ❖ **Update Bias:**
 - ❖ $b(\text{new}) = b(\text{old}) + \eta * \delta$
- ❖ **If $y == t_p$ (No Error):**
 - ❖ No update is needed. The weights and bias remain the same for this training example.
 - ❖ $w_i(\text{new}) = w_i(\text{old})$
 - ❖ $b(\text{new}) = b(\text{old})$

Step 8: Check for Convergence

- ❖ After iterating through all training examples in an epoch, check if any weights or bias were updated.
- ❖ If no updates occurred during an entire epoch, it means the perceptron has learned all patterns correctly and has converged. The training can stop.
- ❖ Otherwise, go back to Step 3 for the next epoch.

Step 9: Termination

- ❖ The algorithm terminates when:
 - ❖ The perceptron converges (no errors for an entire epoch), or
 - ❖ A predefined maximum number of epochs is reached (to prevent infinite loops in non-linearly separable cases).

Gradient Descent

- ❖ Gradient Descent is known as one of the most commonly used **optimization algorithms** to train machine learning models by means of **minimizing errors between actual and expected results**.
- ❖ Further, gradient descent is also used to **train Neural Networks**.
- ❖ In mathematical terminology, Optimization algorithm refers to the task of minimizing/maximizing an objective function $f(x)$ parameterized by x .
- ❖ Similarly, in machine learning, optimization is the task of minimizing the **cost function parameterized by the model's parameters**.
- ❖ The main objective of gradient descent is **to minimize the convex function** using iteration of parameter updates.
- ❖ Once these machine learning models are optimized, these models can be used as powerful tools for Artificial Intelligence and various computer science applications.

What is Gradient Descent or Steepest Descent?

- ❖ Gradient descent was initially discovered by "**Augustin-Louis Cauchy**" in mid of 18th century.
- ❖ Gradient Descent is defined as one of the most commonly used iterative optimization algorithms of machine learning to train the machine learning and **deep learning models**.
- ❖ It helps in finding the **local minimum of a function**.
- ❖ If we move towards a **negative gradient** or away from the gradient of the function at the current point, it will give the **local minimum of that function**.
- ❖ Whenever we move towards a **positive gradient or towards the gradient of the function** at the current point, we will get the **local maximum of that function**.
- ❖ This entire procedure is known as **Gradient Ascent**, which is also known as **steepest descent**.
- ❖ The main objective of using a gradient descent algorithm is to minimize the **cost function** using iteration. To achieve this goal, it performs two steps iteratively:

- ❖ The goal of the gradient descent algorithm is to minimize the given function (say, cost function). To achieve this goal, it performs two steps iteratively:
 1. Compute the gradient (slope), the first-order derivative of the function at that point.
 2. Make a step (move) in the direction opposite to the gradient. The opposite direction of the slope increases from the current point by alpha times the gradient at that point.

$$\begin{aligned} &\text{repeat until convergence} \{ \\ &\quad \theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) \\ &\quad (\text{for } j = 1 \text{ and } j = 0) \\ &\} \end{aligned}$$

- ❖ Alpha is called Learning rate – a tuning parameter in the optimization process. It decides the length of the steps.

What is Cost-function?

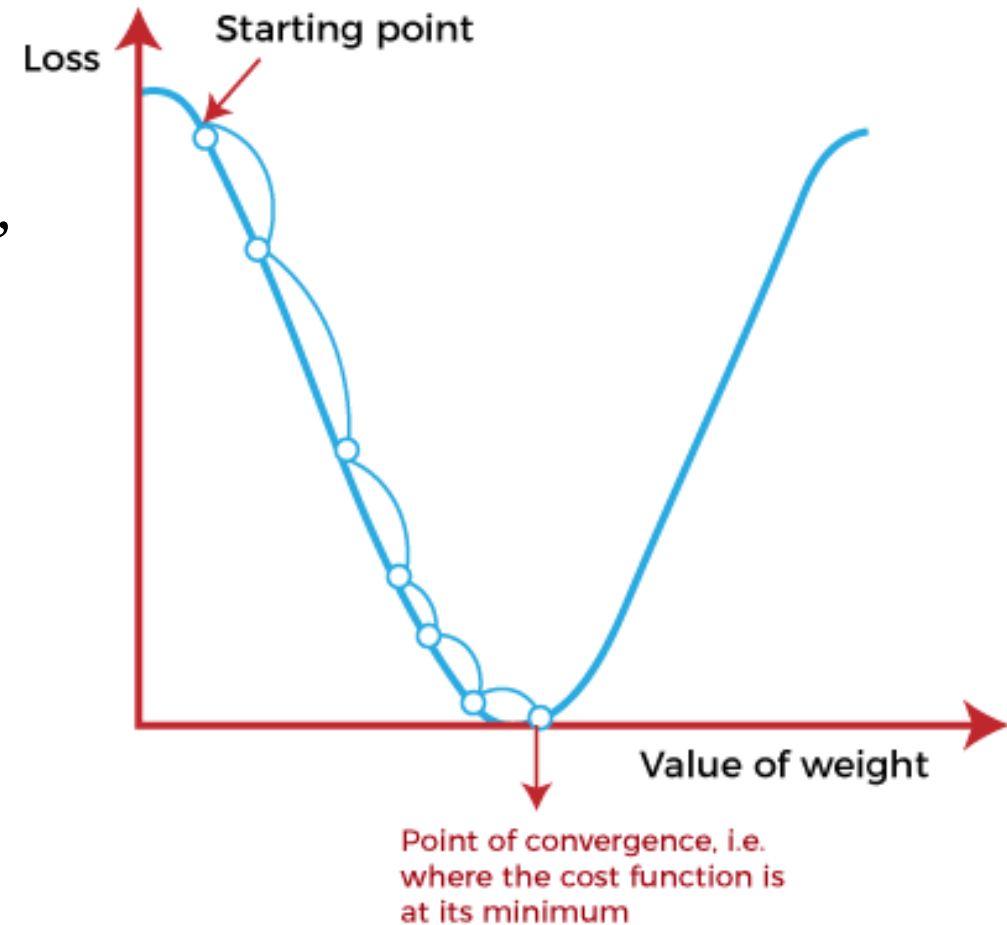
- ❖ The cost function is defined as the **measurement of difference or error between actual values and expected values at the current position** and present in the form of a single real number.
- ❖ It helps to increase and improve machine learning efficiency by providing feedback to this model so that it can minimize error and find the local or global minimum
- ❖ Further, **it continuously iterates along the direction of the negative gradient until the cost function approaches zero.**
- ❖ At this steepest descent point, the model will stop learning further. Although cost function and loss function are considered synonymous, also there is a minor difference between them.
- ❖ The slight difference between the loss function and the cost function is about the error within the training of machine learning models, as loss function refers to the error of one training example, while a cost function calculates the average error across an entire training set.
- ❖ The cost function is calculated after making a hypothesis with initial parameters and modifying these parameters using gradient descent algorithms over known data to reduce the cost function.

How does Gradient Descent work?

- ❖ Before starting the working principle of gradient descent, we should know some basic concepts to find out the slope of a line from linear regression. The equation for simple linear regression is given as:

$$Y=mX+c$$

- ❖ Where 'm' represents the slope of the line, and 'c' represents the intercepts on the y-axis.



- ❖ The starting point(shown in above fig.) is used to evaluate the performance as it is considered just as an arbitrary point.
- ❖ At this starting point, we will derive the first derivative or slope and then use a tangent line to calculate the steepness of this slope. Further, this slope will inform the updates to the parameters (weights and bias).
- ❖ The slope becomes steeper at the starting point or arbitrary point, but whenever new parameters are generated, then steepness gradually reduces, and at the lowest point, it approaches the lowest point, which is called a point of convergence.
- ❖ The main objective of gradient descent is to minimize the cost function or the error between expected and actual.
- ❖ To minimize the cost function, two data points are required:

Direction & Learning Rate

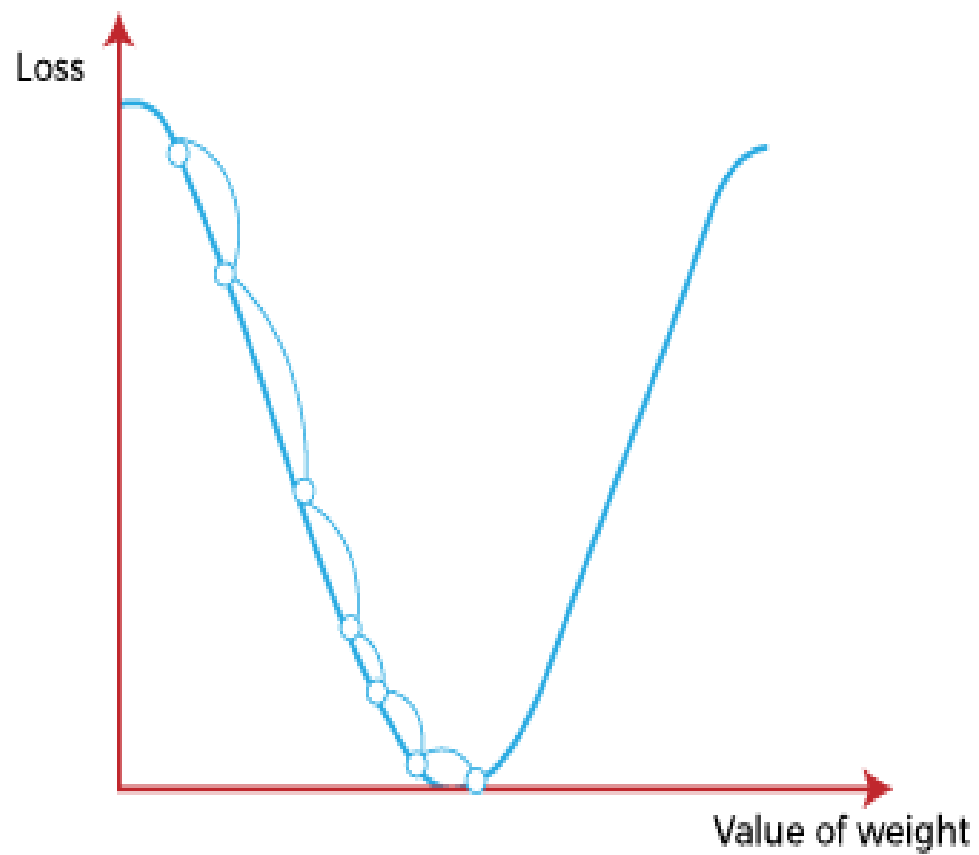
- ❖ These two factors are used to determine the partial derivative calculation of future iteration and allow it to the point of convergence or local minimum or global minimum.

Let's discuss learning rate factors in brief;

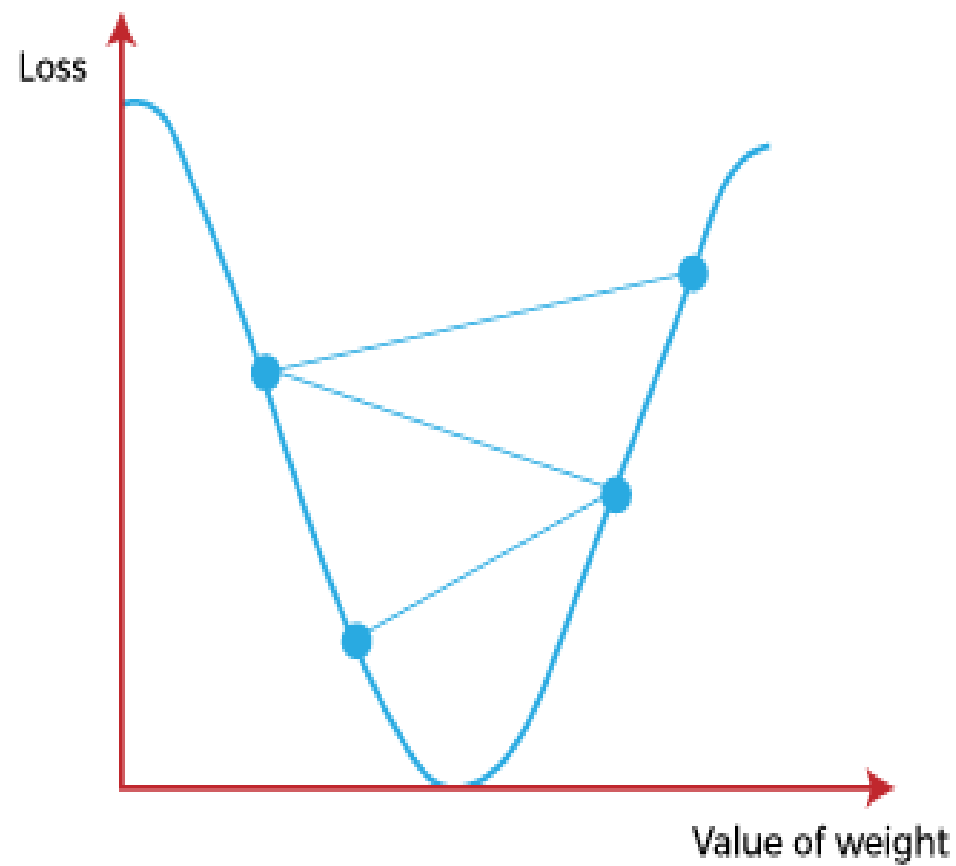
Learning Rate:

- ❖ It is defined as the step size taken to reach the minimum or lowest point.
- ❖ This is typically a small value that is evaluated and updated based on the behavior of the cost function.
- ❖ If the learning rate is high, it results in larger steps but also leads to risks of overshooting the minimum.
- ❖ At the same time, a low learning rate shows the small step sizes, which compromises overall efficiency but gives the advantage of more precision.

Small Learning Rate



Large Learning Rate



Types of Gradient Descent

- ❖ Based on the error in various training models, the Gradient Descent learning algorithm can be divided into Batch gradient descent, stochastic gradient descent, and mini-batch gradient descent.
- ❖ Let's understand these different types of gradient descent:

1. Batch Gradient Descent:

- ❖ Batch gradient descent (BGD) is used to find the error for each point in the training set and update the model after evaluating all training examples.
- ❖ Batch gradient descent is a machine learning optimization algorithm that updates model parameters by computing the gradient of the cost function using the entire training dataset. It's a method for finding the best parameters for a model by iteratively moving in the direction of the steepest descent of the cost function
- ❖ This procedure is known as the training epoch. In simple words, it is a greedy approach where we have to sum over all examples for each update.

Batch Gradient Descent Working Steps:

Step 1: Calculate the gradient:

- ❖ For each iteration, the algorithm computes the gradient of the cost function (which measures the error between predicted and actual values) with respect to the model's parameters using the entire training dataset.

Step 2: Update parameters:

- ❖ The model's parameters are updated using the calculated gradient and a learning rate (a hyperparameter that controls the step size). This step is taken in the opposite direction of the gradient, aiming to minimize the cost function.

Step 3. Repeat:

- ❖ Steps 1 and 2 are repeated until a stopping criterion is met (e.g., reaching a minimum cost or a maximum number of iterations).

Advantages:

❖ Guaranteed convergence:

- ❖ If the cost function is convex, batch gradient descent is guaranteed to converge to the global minimum.

❖ Stable updates:

- ❖ Using the entire dataset for each update provides a stable and accurate estimate of the gradient, leading to more reliable convergence.

Disadvantages:

❖ Slow for large datasets:

- ❖ Processing the entire dataset for each update can be computationally expensive and slow, especially for large datasets.

❖ Not suitable for online learning:

- ❖ Batch gradient descent requires the entire dataset to be available before training, making it unsuitable for online learning scenarios where data arrives sequentially.

❖ Can get stuck in local minima:

- ❖ For non-convex cost functions, batch gradient descent can get stuck in local minima, especially if the learning rate is not chosen carefully.

2. Stochastic gradient descent

- ❖ Stochastic Gradient Descent (SGD) is an iterative optimization algorithm used in machine learning to minimize an objective function, typically a loss function, by updating model parameters. It is a variant of the more general Gradient Descent algorithm.

How SGD Works:

1. **Initialization:** Start with a set of randomly initialized model parameters (weights and biases).
2. **Iteration:**
 - ❖ **For each iteration:**
 - ❖ **Random Sample:** Randomly select a single training example (or a small mini-batch).
 - ❖ **Calculate Gradient:** Compute the gradient of the loss function with respect to the model parameters using only this selected example (or mini-batch).
 - ❖ **Update Parameters:** Adjust the model parameters in the opposite direction of the calculated gradient, scaled by a learning rate (step size). This moves the parameters towards a lower point in the loss function.
 - ❖ **Repeat:** Continue iterating until a stopping criterion is met (e.g., convergence, maximum number of iterations).

Advantages of Stochastic gradient descent:

- ❖ In Stochastic gradient descent (SGD), learning happens on every example, and it consists of a few advantages over other gradient descent.
- ❖ It is easier to allocate in desired memory.
- ❖ It is relatively fast to compute than batch gradient descent.
- ❖ It is more efficient for large datasets.

What is the difference between BGD and SGD?

- ❖ **Batch gradient descent** takes longer to converge since it computes the gradient using the entire training dataset in each iteration.
- ❖ **Stochastic gradient descent**, on the other hand, can converge faster since it updates the model parameters after processing each example, which can lead to faster convergence.

Mini-Batch Gradient Descent

- ❖ Mini-batch gradient descent is an optimization algorithm used to train machine learning models, particularly neural networks. It represents a compromise between Batch Gradient Descent (BGD) and Stochastic Gradient Descent (SGD).

How it works:

Step 1: Divide data into mini-batches:

- ❖ The training dataset is divided into smaller, randomly selected subsets called mini-batches.

Step 2: Calculate gradient for each mini-batch:

- ❖ For each mini-batch, the model's error is calculated, and the gradient of the loss function with respect to the model's parameters is computed.

Step 3: Update parameters:

- ❖ The model's parameters (weights and biases) are updated based on the average gradient calculated from the current mini-batch. This update occurs after processing each mini-batch, rather than after processing the entire dataset (as in BGD) or after each individual data point (as in SGD).

Step 4: Repeat:

- ❖ Steps 1-3 are repeated for all mini-batches in an epoch, and this process is iterated over multiple epochs until the model converges or a stopping criterion is met.

Advantages:

❖ Computational Efficiency:

- ❖ It is more computationally efficient than BGD, especially for large datasets, as it does not require loading the entire dataset into memory for each update. It also allows for parallelization of computations.

❖ Faster Convergence:

- ❖ It provides more frequent updates than BGD, leading to faster convergence in many cases.

❖ Improved Stability:

- ❖ It offers more stable convergence compared to SGD, as the gradient calculated from a mini-batch is less noisy than the gradient from a single data point.

❖ Better Generalization:

- ❖ The inherent randomness introduced by using mini-batches can help the model escape local minima and potentially lead to better generalization performance.

Comparison: Batch vs Mini-Batch vs Stochastic Gradient Descent

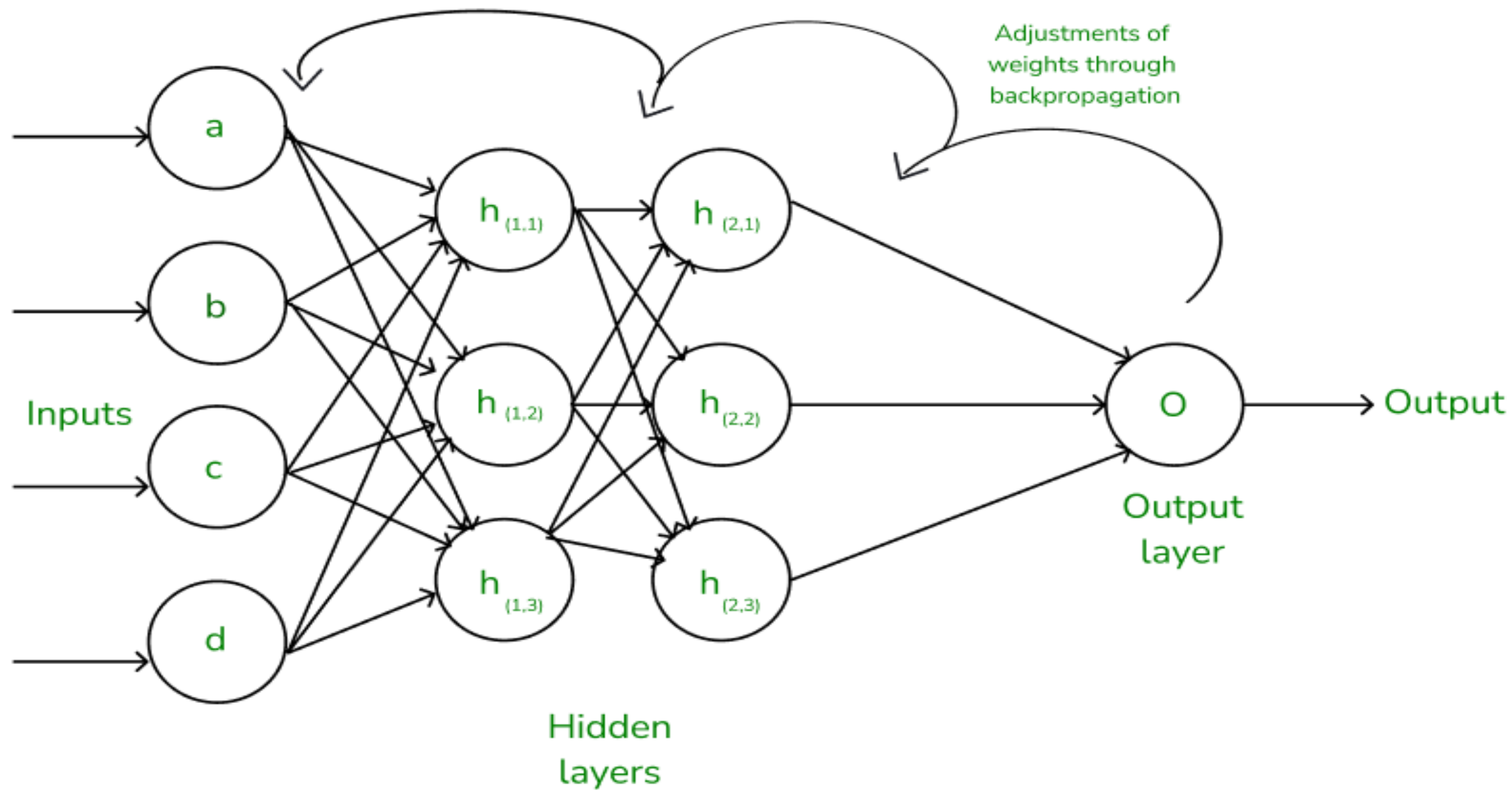
Aspect	Batch Gradient Descent	Stochastic Gradient Descent (SGD)	Mini-Batch Gradient Descent
1. Data Used	Entire dataset	One data point	Small batch (e.g., 32, 64 samples)
2. Speed	Slow (one update per epoch)	Fast (one update per sample)	Fast and balanced (many updates per epoch)
3. Accuracy / Stability	Very stable, smooth convergence	Noisy updates, may oscillate	Balanced accuracy with moderate stability
4. Memory Usage	High (needs full dataset in memory)	Very low (only one sample needed)	Moderate (only mini-batch needed)
5. Best Use Case	Small datasets, offline models	Online learning, streaming data	Deep learning, large datasets
6. Convergence Rate	Slower but steady	Faster but unstable	Faster and more stable than SGD
7. Parallelization	Hard to parallelize	Easy, but limited by sample-by-sample updates	Very easy, ideal for GPU/TPU parallel processing

Backpropagation- Algorithm

- ❖ Backpropagation is a dynamic algorithm in the training of artificial neural networks, enabling them to learn complicated patterns and relationships inside data.
- ❖ The system starts off evolved with an ahead skip, where input statistics traverse through the network, undergoing weighted summations and activation features at each layer.
- ❖ **The computed output is then compared to the real target values, generating a loss that quantifies the gap. In the subsequent backward pass, the gradient of the loss with an appreciation of the network's weights and biases is calculated with the use of the chain rule of calculus.**

The objective of backpropagation is to minimize the error (or loss) between the predicted output of a neural network and the actual target output. This is achieved by systematically adjusting the weights of the network connections to make the network's predictions more accurate over time.

- ❖ In machine learning, backpropagation is an effective algorithm used to train artificial neural networks, especially in feed-forward neural networks.
- ❖ Backpropagation is an iterative algorithm, that helps to minimize the cost function by determining which weights and biases should be adjusted.
- ❖ During every epoch, the model learns by adapting the weights and biases to minimize the loss by moving down toward the gradient of the error.
- ❖ Thus, it involves the two most popular optimization algorithms, such as gradient descent or stochastic gradient descent.
- ❖ Computing the gradient in the backpropagation algorithm helps to minimize the cost function and it can be implemented by using the mathematical rule called chain rule from calculus to navigate through complex layers of the neural network.



Advantages of Using the Backpropagation Algorithm in Neural Networks

- ❖ Backpropagation, a fundamental algorithm in training neural networks, offers several advantages that make it a preferred choice for many machine learning tasks.
- ❖ Here, we discuss some key advantages of using the backpropagation algorithm:
- ❖ **Ease of Implementation:** Backpropagation does not require prior knowledge of neural networks, making it accessible to beginners. Its straightforward nature simplifies the programming process, as it primarily involves adjusting weights based on error derivatives.
- ❖ **Simplicity and Flexibility:** The algorithm's simplicity allows it to be applied to a wide range of problems and network architectures. Its flexibility makes it suitable for various scenarios, from simple feed-forward networks to complex recurrent or convolutional neural networks.
- ❖ **Efficiency:** Backpropagation accelerates the learning process by directly updating weights based on the calculated error derivatives. This efficiency is particularly advantageous in training deep neural networks, where learning features of a function can be time-consuming.

- ❖ **Generalization:** Backpropagation enables neural networks to generalize well to unseen data by iteratively adjusting weights during training. This generalization ability is crucial for developing models that can make accurate predictions on new, unseen examples.
- ❖ **Scalability:** Backpropagation scales well with the size of the dataset and the complexity of the network. This scalability makes it suitable for large-scale machine learning tasks, where training data and network size are significant factors

Importance of Backpropagation

- ❖ **Efficiency:** Backpropagation is much faster and more efficient than computing weight updates manually or randomly.
- ❖ **Foundation:** It is the foundation for training many types of neural networks, from simple ones to deep learning models.

Working of Backpropagation Algorithm

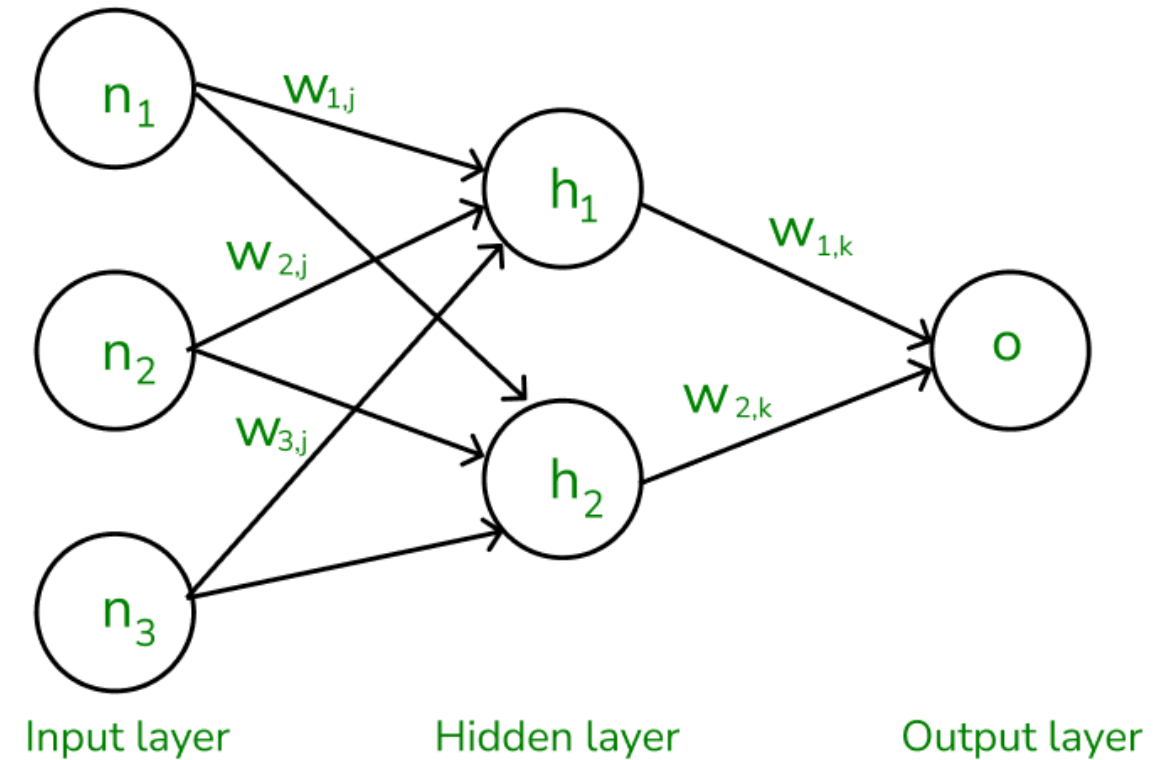
❖ The Backpropagation algorithm works by two different passes, they are:

1. Forward pass
2. Backward pass

How does Forward pass work?

- ❖ In forward pass, initially the input is fed into the input layer. Since the inputs are raw data, they can be used for training our neural network.
- ❖ The inputs and their corresponding weights are passed to the hidden layer. The hidden layer performs the computation on the data it receives.
- ❖ If there are two hidden layers in the neural network, for instance, consider the illustration **fig(a)**, **h1 and h2 are the two hidden layers**, and the **output of h1 can be used as an input of h2**. Before applying it to the activation function, the bias is added.
- ❖ To the weighted sum of inputs, the activation function is applied in the hidden layer to each of its neurons.

- ❖ One such activation function that is commonly used is ReLU can also be used, which is responsible for returning the input if it is positive otherwise it returns zero.
- ❖ By doing this so, it introduces the non-linearity to our model, which enables the network to learn the complex relationships in the data.
- ❖ And finally, the weighted outputs from the last hidden layer are fed into the output to compute the final prediction, this layer can also use the activation function called the softmax function which is responsible for converting the weighted outputs into probabilities for each class.



How does backward pass work?

- ❖ In the backward pass process shows, the error is transmitted back to the network which helps the network, to improve its performance by learning and adjusting the internal weights.
- ❖ To find the error generated through the process of forward pass, we can use one of the most commonly used methods called mean squared error which calculates the difference between the predicted output and desired output. The formula for mean squared error is:

$$\text{Mean squared error} = (\text{predicted output} - \text{actual output})^2$$

- ❖ Once we have done the calculation at the output layer, we then propagate the error backward through the network, layer by layer.
- ❖ The key calculation during the backward pass is determining the gradients for each weight and bias in the network. This gradient is responsible for telling us how much each weight/bias should be adjusted to minimize the error in the next forward pass. The chain rule is used iteratively to calculate this gradient efficiently.
- ❖ In addition to gradient calculation, the activation function also plays a crucial role in backpropagation, it works by calculating the gradients with the help of the derivative of the activation function.

Steps of the Backpropagation Algorithm

1. **Initialization:** Initialize all weights in the network, typically with small random values.
2. **Forward Propagation:** Compute the output of the network for a given input.
3. **Compute Error:** Calculate the error at the output layer using a loss function.
4. **Backpropagate the Error:**
 - ❖ Calculate the gradient of the loss with respect to each weight by moving backward from the output layer to the input layer.
5. **Update Weights:** Adjust the weights in the direction that reduces the error by using the calculated gradients.
6. **Repeat:** Repeat the process for many iterations (epochs) until the network converges to a state where the error is minimized.

Explain about Hyper parameters in Back Propagation?

- ❖ In backpropagation, hyperparameters are the configuration parameters set before training the neural network. These parameters are not learned by the network but significantly influence the training process and the model's performance. Here's an overview of the key hyperparameters used in backpropagation:
- ❖ Hyperparameters in backpropagation are predefined settings that influence the training process and model performance.
- ❖ Key hyperparameters include the learning rate, which controls the step size during weight updates; momentum, which helps smooth updates and avoid local minima; batch size and number of epochs, which determine how data is processed and how long the model trains; and network architecture, such as the number of hidden layers and neurons.

- ❖ Additional factors like activation functions, weight initialization, regularization techniques (e.g., dropout, L2 regularization), and loss functions also play critical roles.
- ❖ Proper tuning of these parameters is essential to balance speed, accuracy, and generalization, often achieved through techniques like grid search or automated optimization.

1. Learning Rate (η):

- ❖ **Determines the step size during weight updates. It controls how much the model adjusts the weights in response to the error gradient.**
- ❖ **Balancing Convergence and Stability:**
- ❖ The learning rate plays a vital role in balancing the speed of convergence and the stability of the training process.
- ❖ **High Learning Rate:** Can lead to faster convergence initially but risks overshooting the optimal solution, causing oscillations or divergence of the loss function.
- ❖ **Low Learning Rate:** Ensures stability and potentially better final model performance but can result in very slow convergence, requiring more training time and computational resources

2. Momentum

- ❖ Momentum is a technique used to accelerate the convergence of optimization algorithms, particularly gradient descent, by **incorporating a fraction of the previous update into the current update**. This helps the algorithm escape local minima and converge to the global minimum more efficiently. Essentially, momentum adds inertia to the optimization process, allowing it to move faster and smoother through the loss landscape.

3. Number of Hidden Layers and Neurons:

- ❖ Specifies the architecture of the network, including the number of hidden layers and the neurons in each layer.
- ❖ **More layers/neurons:** Can capture more complex patterns but increases the risk of overfitting and computational cost.
- ❖ **Fewer layers/neurons:** May lead to underfitting if the network cannot model the data's complexity.

4. Batch Size

- ❖ Batch Size refers to the number of training examples utilized in one forward and backward pass through a neural network before the model's weights are updated. It essentially determines how much data is processed at a time during training.
- ❖ **Small batch size:** Leads to noisy updates but may generalize better.
- ❖ **Large batch size:** Provides more stable updates but requires more memory and may overfit.

5. Number of Epochs:

- ❖ An epoch represents **one complete pass of the entire training dataset through the neural network**. This means that during one epoch, the model processes every single training example, updates its internal parameters (weights and biases) based on the calculated errors, and then is ready to process the entire dataset again in the next epoch.
- ❖ Deep learning models learn iteratively. Each epoch allows the model to refine its understanding of the data by repeatedly adjusting its parameters to minimize the difference between its predictions and the actual target values.
- ❖ **Too many epochs:** Can lead to overfitting.
- ❖ **Too few epochs:** May result in underfitting.

6. Activation Function

- ❖ Activation functions are mathematical functions applied to the output of a neuron in a neural network. Their primary purpose is to introduce non-linearity into the network, enabling it to learn and model complex, non-linear relationships within data.
- ❖ Determines the output of neurons given their input. Common activation functions include ReLU, sigmoid, and tanh
- ❖ Improper choice may lead to vanishing/exploding gradients, hindering effective learning.

7. Weight Initialization

- ❖ Weight initialization in deep learning refers to the process of setting the initial values for the parameters (weights and biases) of a neural network before the training process begins. This initial assignment of values significantly impacts the training dynamics and the final performance of the deep learning model.
- ❖ Poor initialization can lead to gradients becoming extremely small (vanishing) or extremely large (exploding) during backpropagation, hindering the learning process or causing instability.

8. Regularization Parameters

- ❖ Regularization parameters in deep learning are used to prevent overfitting and improve a model's ability to generalize to unseen data.
- ❖ They add a penalty term to the loss function, discouraging the model from learning overly complex or noisy patterns from the training data. By controlling the complexity of the model, regularization parameters help to create models that are more robust and perform better on new, unseen data
- ❖ **L1 Regularization (Lasso):**
- ❖ Adds a penalty proportional to the absolute value of the weights. It can lead to sparse models where some weights are driven to zero, effectively performing feature selection.
- ❖ **L2 Regularization (Ridge):**
- ❖ Adds a penalty proportional to the square of the weights. It shrinks the weights towards zero, but rarely sets them exactly to zero.
- ❖ **Dropout:**
- ❖ Randomly sets a fraction of the neurons in a layer to zero during training, forcing the network to learn more robust features.
- ❖ **Early Stopping:**
- ❖ Stops training before the model overfits the training data, based on performance on a validation set.

❖ 9. Gradient Clipping

- ❖ Gradient clipping is a technique used in deep learning to address the exploding gradient problem, which occurs when the gradients during training become excessively large. This can lead to numerical instability, divergence of the model, and difficulty in converging to an optimal solution.
- ❖ This method directly clips individual gradient values to a specified range (e.g., between -0.5 and 0.5). If a gradient value falls outside this range, it is set to the respective minimum or maximum threshold.

10. Loss Function

- ❖ Loss Function, also known as a cost function or objective function, is a mathematical function that quantifies the discrepancy between a model's predicted output and the actual ground truth label for a given input. It serves as a measure of how poorly the model is performing on the training data.

**Some problems in ANN Optimization and Regularization: Over fitting and Capacity,
Cross Validation, Feature Selection, Regularization, Hyper parameters**

ANN Optimization

- ❖ Artificial Neural Network (ANN) optimization refers to the process of tuning the parameters (weights and biases) in the network to minimize a loss function, typically using algorithms such as stochastic gradient descent (SGD) and its variants (Adam). The main goals are:
 - ❖ Reduce the error between the network's predictions and the actual target values.
 - ❖ Find model parameters that generalize well to unseen data.

Optimization Process:

- ❖ **Forward propagation:** Compute the predicted outputs given the current weights.
- ❖ **Loss calculation:** Measure how far the predictions are from the true values.
- ❖ **Backward propagation:** Calculate gradients of the loss with respect to each weight.
- ❖ **Update weights:** Adjust parameters in the opposite direction of the gradient (using the learning rate).
- ❖ Modern optimizers can adjust learning rates per parameter or use momentum to accelerate convergence and avoid local minima.

Common Regularization Techniques

- ❖ **L1 Regularization (Lasso):** Adds a penalty proportional to the absolute value of weights to the loss function ($\lambda \sum |w|$), promoting sparsity in weights.
- ❖ **L2 Regularization (Ridge):** Adds a penalty proportional to the square of weights ($\lambda \sum w^2$), discouraging large weights.
- ❖ **Dropout:** Randomly deactivates a subset of neurons during training, forcing the network to develop redundant, robust features.
- ❖ **Early stopping:** Monitors validation performance and halts training when performance stops improving, preventing overfitting to training data.
- ❖ **Data augmentation:** Increases the training dataset by applying random transformations (for images: rotation, flipping, etc.), helping the model generalize better.

Overfitting in ANN

- ❖ Overfitting occurs when a neural network learns patterns and details—including noise—from the training data to such an extent that its performance on new, unseen data degrades. Signs of overfitting include:
 - ❖ Low training error but high validation/test error.
 - ❖ Model memorizes specific examples instead of learning general patterns.

How Regularization Prevents Overfitting:

- ❖ By penalizing complexity (large weights, unnecessary neurons), regularization helps the model favor simpler, more generalizable solutions.
- ❖ Dropout and data augmentation introduce randomness, which prevents the network from relying too heavily on specific features.
- ❖ Early stopping ensures training halts before the network begins to learn noise.

ANN Optimization and Regularization: Capacity

- ❖ Capacity in the context of Artificial Neural Networks (ANNs) refers to the network's ability to model a wide variety of functions. It is largely determined by the architecture—the number of layers and the number of neurons in each layer. Here's how capacity, optimization, and regularization interrelate:

Optimization and Capacity

- ❖ ANN optimization seeks the best set of parameters (weights and biases) for a given network architecture to minimize prediction error.
- ❖ Capacity determines how complex a function the network can fit. A higher-capacity network (more layers/neurons) can represent more intricate patterns, but is also at a higher risk of memorizing the training data (overfitting), especially when the training set is small or noisy.
- ❖ During optimization, if the model's capacity is too low (too few parameters), it may underfit—unable to capture the underlying trend (high bias). If capacity is too high, it may overfit by learning noise and outliers, not general trends (high variance)

Regularization and Capacity

- ❖ Regularization techniques are essential for managing model capacity, especially in high-capacity networks. They help control overfitting by discouraging the model from learning noise or unnecessary complexity.
- ❖ L1 and L2 regularization add penalties to large weights, effectively limiting the model's effective capacity even in large architectures.
- ❖ Dropout randomly disables parts of the network during training, making the network less reliant on specific nodes and distributing learning across multiple features. This narrows down the effective capacity for a given input.
- ❖ Early stopping halts training before the model becomes too specialized to the training data, thus simplifying the function learned.
- ❖ Data augmentation increases the variety of training data, ensuring that the network's high capacity is used to learn robust features rather than memorizing particular examples

Cross Validation in ANN

- ❖ Cross-validation is a model evaluation technique. It is used to assess how well your ANN model will **generalize to an independent dataset** (i.e., data it hasn't seen before). Instead of training the model once and testing it on a single test set, we split the data multiple times and train/test the model multiple times.
- ❖ Cross validation is a resampling procedure used to assess a model's ability to predict new, unseen data. It is especially important in neural networks, as these models are prone to overfitting due to their high capacity and flexibility.

Why is Cross-Validation Needed in ANN?

- ❖ Training neural networks typically requires a lot of data. If we train on one part and test on another, our results might depend too much on how the data was split. Cross-validation:
- ❖ Reduces bias in model evaluation.
- ❖ Gives a better estimate of how the ANN performs.
- ❖ Helps prevent overfitting (learning the training data too well but failing on new data).
- ❖ Helps in hyperparameter tuning (e.g., learning rate, batch size, number of neurons).

How Cross Validation Works in ANN

❖ Data Splitting:

- ❖ The data is divided into 'k' roughly equal-sized folds or subsets.
- ❖ The model is trained k times: each time, one fold is used for validation, and the remaining (k-1) are used for training.

❖ Training and Evaluation:

- ❖ For each fold, a fresh copy of the ANN is initialized and trained.
- ❖ After training, performance is measured on the validation fold.
- ❖ This process is repeated for each fold, ensuring each subset gets used for validation once.

❖ Result Aggregation:

- ❖ The final performance metric (accuracy, loss, etc.) is calculated by averaging metrics across all k validation folds.
- ❖ This gives a more robust estimate than a single train/test split.

Types of Cross Validation Used in ANN

❖ K-Fold Cross-Validation (Most Common):

- ❖ K-fold cross-validation is a technique used in ML / Deep Learning to evaluate the performance of a model by **dividing the dataset into K subsets (or folds)**, training the model on K-1 folds, and testing it on the remaining fold.
- ❖ This process is repeated K times, with each fold used as the test set once. The final performance metric is the average of the results from all K iterations, providing a more robust estimate of the model's generalization ability than a single train/test split.

Steps:

1. Divide your data into K equal parts (folds).
2. For each iteration:
 1. Use K-1 folds for training.
 2. Use the remaining 1 fold for validation.
3. Repeat this K times (each fold gets a chance to be the validation set).
4. Average the results (accuracy, loss, etc.).

Example (K=5):

- ❖ Split data into 5 parts: A, B, C, D, E
- ❖ Iteration 1: Train on B+C+D+E, validate on A
- ❖ Iteration 2: Train on A+C+D+E, validate on B
- ❖ and so on.
- ❖ Compute average validation performance.
- ❖ **Advantage:** All data is used for both training and validation.

Benefits of K-fold cross-validation:

Reduced Variance:

- ❖ By using all data for both training and validation, it provides a more reliable estimate of the model's performance than a single train/test split.

Handles Imbalanced Data:

- ❖ It can be particularly useful when dealing with imbalanced datasets, where one class has significantly fewer instances than others, by ensuring each fold has a representative sample of all classes.

Stratified K-Fold Cross-Validation

- ❖ Stratified K-Fold Cross-Validation is an enhanced version of K-Fold Cross-Validation, particularly valuable for classification problems, especially those with imbalanced datasets.
- ❖ Especially important if classes are imbalanced (e.g., 90% class A, 10% class B).

Leave-One-Out Cross-Validation (LOOCV)

- ❖ Leave-One-Out Cross-Validation (LOOCV) is a specific type of k-fold cross-validation where the number of folds, 'k', is equal to the number of data points, 'n', in the dataset.
- ❖ Extreme case of K-Fold where $K = \text{number of data points}$.
- ❖ Each sample is used once as the validation data, all others are used for training.
- ❖ Very computationally expensive for ANN, rarely used.

Repeated K-Fold Cross-Validation:

- ❖ Repeated k-fold cross-validation is an extension of the standard k-fold cross-validation technique used for evaluating machine learning models. It involves performing the k-fold cross-validation procedure multiple times, with a different random shuffling of the data and subsequent splitting into k-folds for each repetition.
- ❖ Repeat the K-Fold process multiple times (with different splits).
- ❖ Reduces variance of the validation estimate even more.

K-Fold Cross-Validation of an Artificial Neural Network (ANN)

- 1. #Importing Libraries**
- 2. from sklearn.model_selection import KFold**
- 3. import numpy as np**
- 4. from tensorflow.keras.models import Sequential**
- 5. from tensorflow.keras.layers import Dense**
- 6. from tensorflow.keras.optimizers import Adam**

#Creating Sample Data

- 1. X = np.random.rand(1000, 20) # 1000 samples, 20 input features**
- 2. y = np.random.randint(2, size=(1000, 1)) # 1000 binary labels (0 or 1)**

1. **# Set up 5-fold cross-validation**
2. **kfold = KFold(n_splits=5, shuffle=True, random_state=42)**
3. **fold_no = 1**
4. **acc_per_fold = []**
5. **for train_idx, val_idx in kfold.split(X):**
6. **model = Sequential([**
7. **Dense(64, activation='relu', input_shape=(20,)),**
8. **Dense(32, activation='relu'),**
9. **Dense(1, activation='sigmoid')**
10. **])**
11. **model.compile(optimizer=Adam(), loss='binary_crossentropy',**
 metrics=['accuracy'])

1. `# Train the model`
2. `model.fit(X[train_idx], y[train_idx], epochs=10, batch_size=32, verbose=0)`
1. `# Evaluate on validation set`
2. `scores = model.evaluate(X[val_idx], y[val_idx], verbose=0)`
3. `print(f"Fold {fold_no} - Validation Accuracy: {scores[1]*100:.2f}%")`
4. `acc_per_fold.append(scores[1] * 100)`
5. `fold_no += 1`
6. `print(f"\nAverage Validation Accuracy: {np.mean(acc_per_fold):.2f}%")`

```
Fold 1 - Validation Accuracy: 55.50%  
Fold 2 - Validation Accuracy: 51.00%  
Fold 3 - Validation Accuracy: 54.00%  
Fold 4 - Validation Accuracy: 55.00%  
Fold 5 - Validation Accuracy: 48.00%
```

```
Average Validation Accuracy: 52.70%
```

Feature Selection in ANN

- ❖ Feature Selection is the process of selecting the most important input variables (features) from a dataset that are most relevant to the prediction or classification task. In Artificial Neural Networks (ANNs), good feature selection can improve performance, reduce training time, and help avoid overfitting.
- ❖ By reducing the number of input variables, feature selection helps to build simpler, faster, and more accurate models.

Why is Feature Selection Important for ANNs?

- ❖ Neural networks, especially deep ones, are powerful at finding complex patterns, but they can be sensitive to the quality of the input data. Including irrelevant or redundant features can lead to several problems:
- ❖ **Overfitting:** A model that is too complex might memorize the training data, including the noise, and perform poorly on new, unseen data. Irrelevant features add noise, making the model more likely to overfit.

- ❖ **Curse of Dimensionality:** As the number of features increases, the amount of data required to achieve a good model performance grows exponentially. This phenomenon, known as the curse of dimensionality, can make training a model with a large number of features computationally expensive and difficult.
- ❖ **Increased Training Time:** More features mean more weights and biases in the neural network, which increases the number of computations required during training. This can significantly slow down the training process.
- ❖ **Reduced Interpretability:** ANNs are often considered "black boxes." While it's already hard to understand how they make decisions, having fewer input features can make it easier to interpret the model's outputs and understand which factors are most influential.

Methods of Feature Selection in ANN

- ❖ Feature selection techniques can be broadly categorized into three types
- ❖ **Filter Methods:** These methods select features based on their statistical properties without involving a specific machine learning algorithm. They are generally computationally cheaper and faster than other methods. The chosen features are independent of the model.
 - ❖ **Correlation:** Features with a high correlation with the target variable are often considered more relevant.
 - ❖ **ANOVA (Analysis of Variance):** A statistical test that determines if there is a significant difference between the means of two or more groups, which can be used to assess the relationship between a feature and the target variable.
 - ❖ **Chi-squared test:** Used to test for the relationship between categorical variables.
 - ❖ **Mutual Information:** Measures the dependency between two variables. It can capture non-linear relationships, unlike correlation.

- ❖ **Wrapper Methods:** These methods use a specific machine learning model (in this case, an ANN) to evaluate subsets of features. They train a model on different feature subsets and select the one that results in the best performance (e.g., highest accuracy). While they can produce better results than filter methods, they are computationally more expensive.
- ❖ **Forward Selection:** Starts with an empty set of features and iteratively adds the feature that provides the greatest improvement to the model's performance until a stopping criterion is met.
- ❖ **Backward Elimination:** Starts with all features and iteratively removes the least significant feature until a stopping criterion is met.
- ❖ **Recursive Feature Elimination (RFE):** A greedy algorithm that repeatedly builds a model, ranks features based on their importance, and removes the least important ones. This process is repeated until the desired number of features is reached.

- ❖ **Embedded Methods:** These methods perform feature selection as part of the model training process itself. They combine the advantages of filter and wrapper methods by having a built-in mechanism for feature importance.
- ❖ **Regularization:** Techniques like Lasso (L1 regularization) can be incorporated into the ANN's loss function. Lasso regularization adds a penalty equal to the absolute value of the magnitude of the weights. This penalty can force the weights of less important features to become zero, effectively removing them from the model.
- ❖ **Tree-based methods:** While not strictly part of an ANN, algorithms like Random Forests and Gradient Boosting Machines can be used to pre-select features. These algorithms have a built-in feature importance score (e.g., Gini impurity or information gain) that can be used to rank and select features before training an ANN.
- ❖ **LIME (Local Interpretable Model-agnostic Explanations) or SHAP (SHapley Additive exPlanations):** These are techniques for explaining the predictions of any machine learning model. They can be used to understand which features have the most influence on an ANN's output, thus providing a basis for feature selection

Feature Selection Techniques for ANN

❖ Using Feature Importance:

- ❖ Feature importance helps identify which input features are most useful for predicting the output. By using models like Random Forest or XGBoost, you can rank features based on how much they contribute to decision-making. Then, you can select the top-ranked features and feed them into your Artificial Neural Network (ANN) to improve performance and reduce complexity.

❖ Using Correlation Matrix:

- ❖ When building an Artificial Neural Network (ANN), it's important **to avoid redundant features that carry similar information**. One common way to detect and remove such redundancy is by using a correlation matrix.

❖ Using Autoencoders:

- ❖ Autoencoders are a type of neural network used to compress and reconstruct data. You can use them to automatically learn the most important features (called latent features), and then feed those compressed features into another model like an ANN for classification or prediction.

PCA (Principal Component Analysis):

- ❖ Principal Component Analysis (PCA) is a powerful technique used to reduce the number of input features in your dataset by transforming them into a smaller set of new features (principal components) that still capture the most important patterns (variance) in the data.
- ❖ Although PCA doesn't select the best original features (like feature selection), it transforms them into a smaller, more useful set — which can be used as input to an Artificial Neural Network (ANN).

Regularization in Artificial Neural Networks (ANN)

- ❖ Regularization is a set of techniques used in Artificial Neural Networks (ANNs) to prevent a model from overfitting the training data and to improve its ability to generalize to new, unseen data.
- ❖ Overfitting occurs when a model learns not only the underlying patterns in the training data but also the noise and random fluctuations, leading to a model that performs exceptionally well on the training set but poorly on a validation or test set.
- ❖ Regularization techniques work by adding a penalty term to the model's loss function. This penalty discourages the model from becoming too complex or having excessively large parameter values (weights and biases), which are common characteristics of overfit models.
- ❖ By constraining the model's complexity, regularization encourages it to find a simpler, more generalized solution that is more likely to be representative of the true data distribution.

Regularization Techniques in ANNs

- ❖ Artificial Neural Networks are powerful and flexible — they can learn very complex patterns. But if not controlled, they may also learn noise and irrelevant patterns in the training data.
- ❖ Regularization forces the model to learn only the essential patterns, improving generalization on unseen data.

L1 Regularization (Lasso):

- ❖ L1 Regularization, also called Lasso (Least Absolute Shrinkage and Selection Operator), is a technique that adds a penalty to the loss function of a neural network to reduce overfitting and encourage sparsity.
- ❖ It penalizes the model for having large weight values by adding the sum of the absolute values of the weights to the loss function.

L2 Regularization (Ridge):

- ❖ L2 Regularization, also known as Ridge Regularization, is a technique used in Artificial Neural Networks (ANNs) to prevent overfitting by penalizing large weights.
- ❖ Instead of removing features like L1, it shrinks all weights by adding a penalty term based on the square of the weights to the loss function.

Dropout

- ❖ Dropout is a regularization technique used in ANN to prevent overfitting. During training, it randomly disables (drops out) a percentage of neurons in a layer — meaning their output is ignored for that training step.
 - ❖ Makes the network not rely too much on any one neuron.
 - ❖ Encourages the model to learn redundant, robust representations.
 - ❖ Prevents co-adaptation, where neurons work well together only in specific combinations.

Early Stopping

- ❖ Early Stopping is a regularization technique used to prevent overfitting in neural networks by stopping the training process automatically when the model's performance on validation data stops improving.

“To succeed in your mission, you must have single-minded devotion to your goal.”

A. P. J. Abdul Kalam



A red pushpin is pinned to the top center of a yellow sticky note.

Thank
You!